

RELATÓRIO TÉCNICO

Sprint 3 — DevSecOps

Disciplina de Cybersecurity

PROJETO RIVA

Inteligência Competitiva Automotiva — Ford + FIAP 2026 Challenge

Pipeline DevSecOps	Secret scan, SAST, SCA, IaC/Container scan e Dependency Review no CI/CD
Infraestrutura	Imagem Docker endurecida (não-root) e Actuator isolado em produção
Compliance	STRIDE revisado + OWASP ASVS / API Top 10 / Mobile Top 10 + LGPD
Observabilidade	Prometheus + Grafana + Loki, 11 alertas e runbook de resposta a incidentes
Qualidade	156 testes automatizados — 100% de aprovação

Identificação

Integrantes do grupo

Nome	RM
Beatriz Vieira de Novais	RM554746
Guilherme Abe	RM554743
Gustavo Ruiz Vieira Paulino	RM554779
Mariana Neugebauer Dourado	RM550494
Victor Pacifico Dias	RM558017

Repositório

Código-fonte do backend: github.com/FORD-FIAP/rivalA-backend

Branch desta entrega: feature/sprint3-devsecops

Pull Request desta entrega (pipeline + evidências): github.com/FORD-FIAP/rivalA-backend/pull/24

Sumário

Identificação

[Integrantes do grupo](#)

[Repositório](#)

Contexto — evolução das sprints anteriores

Atividade 1 — Pipeline DevSecOps Integrado

[1.1 Desenho do pipeline](#)

[1.2 Etapas e o risco de cada uma](#)

[1.3 Gatilhos, saídas e integração](#)

[1.4 Execução no projeto Ford](#)

Atividade 2 — Segurança em Código e Infraestrutura

[2.1 Segurança no código](#)

[2.2 MQTT/TLS para IoT](#)

[2.3 IaC Security](#)

[2.5 Auditoria interna e correções](#)

Atividade 3 — Compliance, Riscos e Segurança Contínua

[3.1 Riscos — STRIDE + DevSecOps](#)

[3.2 Mapeamento com normas e boas práticas](#)

[3.3 Plano de segurança contínua](#)

[3.4 Checklist de conformidade consolidado](#)

Atividade 4 — Observabilidade, Monitoramento e Resposta

[4.1 Plano de monitoramento](#)

[4.2 Plano de resposta a incidentes](#)

[4.3 Evidências coletadas](#)

Apêndice A — Evidências por atividade

Apêndice B — Rodar tudo localmente

Apêndice C — Glossário

Contexto — evolução das sprints anteriores

Nas sprints anteriores de Cybersecurity a segurança do backend RIVA foi implementada como cinco blocos de defesa em profundidade (validação de entrada, autenticação/RBAC, proteção de APIs, criptografia/LGPD e logging/auditoria), documentada no arquivo SECURITY.md, com 156 testes automatizados e mapeamento contra o OWASP Top 10.

Esta sprint evolui esse trabalho para um modelo DevSecOps: a segurança deixa de ser “um documento mais código” e passa a ser parte contínua do ciclo — verificada a cada commit no pipeline, monitorada em tempo de execução e com procedimento formal de resposta a incidentes.

Ambiente de referência: repositório FORD-FIAP/rivaIA-backend, branches develop (homologação) e production (produção), deploy via SSH e PostgreSQL hospedado na Azure.

#	Atividade	Peso	Evidências no repositório
1	Pipeline DevSecOps Integrado	3,0	.github/workflows/security.yml, .github/dependabot.yml, .gitleaks.toml
2	Segurança em Código e Infraestrutura	2,5	Dockerfile, .dockerignore, src/main/java/**, SECURITY.md
3	Compliance, Riscos e Segurança Contínua	2,5	Seção 3 deste documento + SECURITY.md
4	Observabilidade, Monitoramento e Resposta	2,0	observability/**, docs/RUNBOOK_RESPOSTA_INCIDENTES.md

Nota total: média ponderada das quatro atividades (0 a 10 cada).

ATIVIDADE 1

Pipeline DevSecOps Integrado

PESO 3,0

Objetivo: demonstrar como a segurança é incorporada ao pipeline de desenvolvimento, desde o commit até o deploy.

1.1 Desenho do pipeline

O pipeline de segurança (arquivo `.github/workflows/security.yml`) roda em paralelo ao CI de build e teste já existente. Fluxo:

Desenvolvedor

↓ *git push / pull request*

CI - Build & Test

develop.yml / production.yml
mvnw test + package · 156 testes

DevSecOps - Security Pipeline

- 1. Secret Scanning** — Gitleaks (histórico completo)
 - 2. SAST** — Semgrep (p/java, security-audit, secrets, owasp-top-ten)
 - 3. SCA** — OWASP Dependency-Check + Trivy fs (CVE em dependências)
 - 4. IaC & Container Scan** — Trivy config (Dockerfile/compose/CI) + Trivy image
 - 5. Dependency Review** — alerta (warn-only) sobre dependência vulnerável nova no PR
- Saídas: SARIF → aba Security + relatórios (artifacts)

↓ *merge aprovado*

Deploy via SSH

develop → homologação
production → produção → Runtime + Observabilidade (Atividade 4)

Gatilhos extra: Dependabot (semanal) abre PRs · cron semanal reexecuta a varredura

1.2 Etapas e o risco que cada uma reduz

Etapa	Ferramenta (OSS, sem conta)	O que detecta	Risco reduzido (STRIDE / OWASP)	Gate
Secret Scanning	Gitleaks 8.21	Chaves, tokens e senhas no código e em todo o histórico git	Information Disclosure; A02:2021; vazamento de JWT_SECRET / AES_ENCRYPTION_KEY / DB_PASSWORD	Bloqueia (exit-code 1); falsos-positivos de dev tratados no <code>.gitleaks.toml</code>

Etapa	Ferramenta (OSS, sem conta)	O que detecta	Risco reduzido (STRIDE / OWASP)	Gate
SAST	Semgrep OSS (p/java, security-audit, secrets, owasp-top-ten)	Injection, cripto fraca, deserialização insegura, path traversal, uso incorreto do Spring Security	Tampering / Elevation; A03, A02, A08	Relatório + SARIF (entra na triagem; não bloqueia merge)
SCA	Trivy fs (CLI) + OWASP Dependency-Check	CVEs conhecidas nas dependências transitivas do Maven	Elevation of Privilege; A06:2021 (Vulnerable & Outdated Components)	Relatório + SARIF (CRITICAL/HIGH); triagem humana
IaC & Container Scan	Trivy config + Trivy image (CLI)	Dockerfile / compose / workflows inseguros; CVEs no SO-base da imagem	Tampering; A05:2021 (Security Misconfiguration)	Relatório + SARIF; revisão obrigatória em CRITICAL
Dependency Review	actions/dependency-review-action (warn-only)	Dependência nova vulnerável ou com licença incompatível introduzida no PR	A06:2021; supply chain	Comentário/resumo no PR (requer Dependency graph habilitado)
Dependabot	GitHub nativo	Versões desatualizadas / advisories (Maven, Actions, Docker)	A06:2021; manutenção contínua	Abre PR automático semanal → passa pelo pipeline

Modo de operação: o Secret Scanning bloqueia (segredo no repositório é parada obrigatória). Os demais scanners rodam em modo informativo nesta entrega — publicam achados (artifacts + aba Security) e o gate é a revisão humana. Para virar gate automático, remover os continue-on-error / exit-code 0 dos jobs desejados e habilitar Dependency graph + Code scanning nas configurações do repositório.

1.3 Gatilhos, saídas e integração

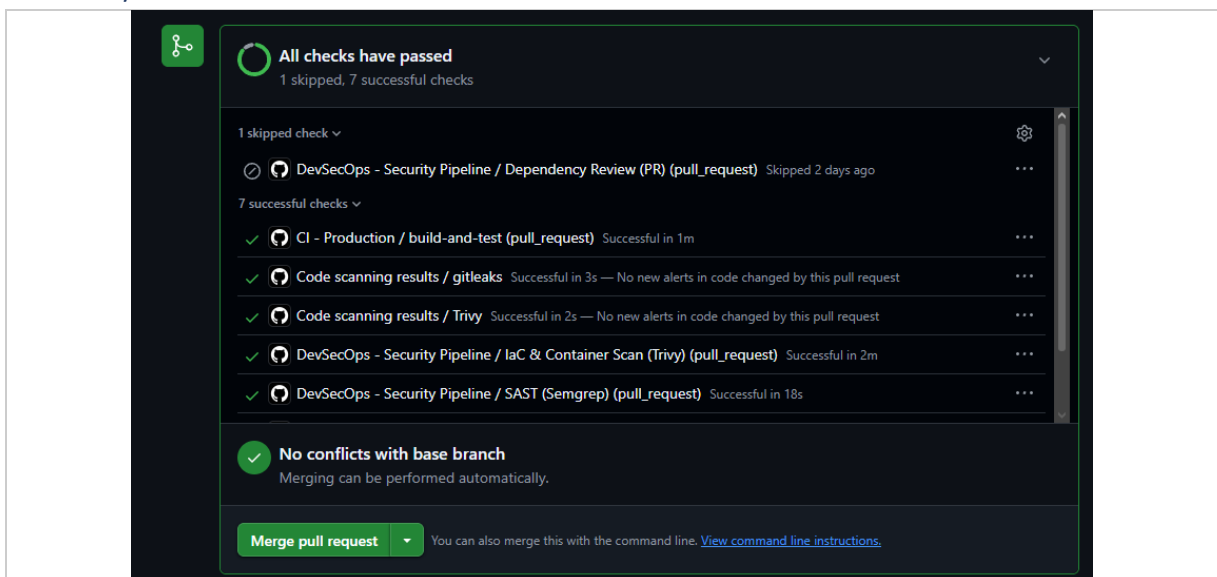
- Gatilhos: push e pull_request em main/develop/production, agenda semanal (segunda 06:00 UTC) e execução manual (workflow_dispatch).
- Saídas: cada job publica um artifact (relatório) e envia SARIF para a aba Security do GitHub (permissão security-events: write), com continue-on-error para não derrubar o build caso o Code Scanning não esteja habilitado.
- Relação com o CI existente: develop.yml / production.yml continuam responsáveis por build + teste; o security.yml roda em paralelo. O deploy via SSH só ocorre após merge, que exige CI verde e revisão dos achados.
- Segredo opcional do CI: NVD_API_KEY acelera o download da base de CVEs do Dependency-Check (Settings → Secrets and variables → Actions).

1.4 Como seria executado no projeto Ford

- Dev abre PR para develop. Rodam CI - Develop e DevSecOps - Security Pipeline.
- Gitleaks sinaliza segredos (bloqueia o job); Semgrep, SCA e Trivy publicam achados para triagem.
- Dependency Review comenta dependências vulneráveis novas no PR (warn-only, dormente até habilitar Dependency graph).
- O revisor tria os achados: corrige agora, cria issue de acompanhamento, ou registra risco aceito.
- Merge em develop → deploy automático em homologação.
- PR develop → production repete o pipeline; após aprovado, deploy em produção via SSH.
- Semanalmente: Dependabot abre PRs de atualização e o security.yml reexecuta a varredura completa.

Evidências: print da aba Actions com os jobs verdes e print do Code scanning (abaixo); os relatórios de cada job (gitleaks-report, semgrep-report, sca-reports, trivy-reports) ficam como artifacts da execução do workflow.

EVIDÊNCIA / SCREENSHOT



Evidência — PR #24: todos os checks do pipeline de segurança passando (Secret Scanning, SAST, SCA, IaC & Container Scan).

EVIDÊNCIA / SCREENSHOT

☐	☒	0 Open	☒	75 Closed		Closed as ▾	Language ▾	Tool ▾	Rule ▾	Severity ▾	Sort ▾
☐	☒				Spring Security: Spring Security: Security policy bypass and information disclosure due to unwritten HTTP headers	Critical	Library				refs/pull/24/merge
					#71 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-security-web-6.3...:1						
☐	☒				tomcat: RCE due to TOCTOU issue in JSP compilation	Critical	Library				refs/pull/24/merge
					#50 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				org.apache.tomcat/tomcat: Apache Tomcat: Unauthorized resource access via FORM authentication bypass	Critical	Library				refs/pull/24/merge
					#49 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				tomcat: Apache Tomcat: Authentication bypass via limited replay attack in DIGEST authenticator	Critical	Library				refs/pull/24/merge
					#48 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				Apache Tomcat: Apache Tomcat: Security constraint bypass due to improper access control	Critical	Library				refs/pull/24/merge
					#47 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				tomcat-coyote: tomcat: Improper Authorization allows security bypass	Critical	Library				refs/pull/24/merge
					#46 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				tomcat-coyote: Apache Tomcat: Authentication bypass via digest authentication	Critical	Library				refs/pull/24/merge
					#45 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				tomcat-coyote: Apache Tomcat: HTTP/2 request headers not validated	Critical	Library				refs/pull/24/merge
					#44 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				tomcat: Potential RCE and/or information disclosure and/or information corruption with partial PUT	Critical	Library				refs/pull/24/merge
					#43 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/tomcat-embed-core-10.13...:1						
☐	☒				Spring Security: Spring Security: Security policy bypass and information disclosure due to unwritten HTTP headers	Critical					refs/pull/24/merge
					#34 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				tomcat: RCE due to TOCTOU issue in JSP compilation	Critical					refs/pull/24/merge
					#12 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				org.apache.tomcat/tomcat: Apache Tomcat: Unauthorized resource access via FORM authentication bypass	Critical					refs/pull/24/merge
					#11 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				tomcat: Apache Tomcat: Authentication bypass via limited replay attack in DIGEST authenticator	Critical					refs/pull/24/merge
					#10 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						

Evidência — Code scanning do PR #24: 75 vulnerabilidades encontradas pelo Trivy (Tomcat, Spring Security, Spring Framework, Spring Data, Spring Boot), todas fechadas como corrigidas após o upgrade de dependências (§2.4).

EVIDÊNCIA / SCREENSHOT

☐	☒				tomcat: RCE due to TOCTOU issue in JSP compilation	Critical					refs/pull/24/merge
					#12 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				org.apache.tomcat/tomcat: Apache Tomcat: Unauthorized resource access via FORM authentication bypass	Critical					refs/pull/24/merge
					#11 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				tomcat: Apache Tomcat: Authentication bypass via limited replay attack in DIGEST authenticator	Critical					refs/pull/24/merge
					#10 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				Apache Tomcat: Apache Tomcat: Security constraint bypass due to improper access control	Critical					refs/pull/24/merge
					#9 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				tomcat-coyote: tomcat: Improper Authorization allows security bypass	Critical					refs/pull/24/merge
					#8 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				tomcat-coyote: Apache Tomcat: Authentication bypass via digest authentication	Critical					refs/pull/24/merge
					#7 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				tomcat-coyote: Apache Tomcat: HTTP/2 request headers not validated	Critical					refs/pull/24/merge
					#6 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				tomcat: Potential RCE and/or information disclosure and/or information corruption with partial PUT	Critical					refs/pull/24/merge
					#5 closed as fixed 2 days ago • Detected by Trivy in pom.xml:1						
☐	☒				org.springframework: Spring Framework: Cross-site scripting (XSS) via incorrect JavaScript escaping	High	Library				refs/pull/24/merge
					#75 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-webmvc-6.1.14.jar:1						
☐	☒				spring-framework: Spring Framework: Denial of Service when resolving static resources	High	Library				refs/pull/24/merge
					#74 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-webmvc-6.1.14.jar:1						
☐	☒				spring-framework: Spring Framework: Denial of Service via specially crafted SpEL expressions	High	Library				refs/pull/24/merge
					#73 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-expression-6.1.14...:1						
☐	☒				org.springframework/spring-core: Spring Framework Annotation Detection Vulnerability	High	Library				refs/pull/24/merge
					#72 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-core-6.1.14.jar:1						
☐	☒				spring-security-core: Spring Security BCryptPasswordEncoder does not enforce maximum password length	High	Library				refs/pull/24/merge
					#70 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-security-crypto-6...:1						
☐	☒				Spring Data Commons: Spring Data Commons: Denial of Service due to cache exhaustion via attacker-supplied strings	High	Library				refs/pull/24/merge
					#69 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-data-commons-3.3...:1						
☐	☒				Spring Boot: Spring Boot: Arbitrary Code Execution and Session Hijacking via predictable temporary directory	High	Library				refs/pull/24/merge
					#68 closed as fixed 2 days ago • Detected by Trivy in app/.../lib/spring-boot-3.3.5.jar:1						

Continuação da lista de achados corrigidos.

ATIVIDADE 2

Segurança em Código e Infraestrutura

PESO 2,5

Objetivo: evidenciar práticas de segurança aplicadas diretamente no código e na infraestrutura.

2.1 Segurança no código — o que já existe e o que foi reforçado nesta sprint

Tema	Implementação	Arquivo / classe	Origem
Criptografia local	AES-256-GCM (autenticada) no User.email; blind index HMAC-SHA256 (email_hash) para busca em coluna cifrada; BCrypt strength 12 nas senhas	crypto/AesEncryptor.java, crypto/EmailHasher.java	sprint anterior
Hardening de API — rate limit	Bucket4j: 60 req/min geral, 10 req/min em /auth/login; 429 + Retry-After; WARN a 80%	security/filter/RateLimitFilter.java	sprint anterior
Hardening de API — validação de entrada	4 camadas: @NotBlank / @Size / @Pattern + @SafeText (análise semântica de XSS, SQLi, command injection, path traversal)	util/InputSanitizer.java, validation/SafeTextValidator.java, dto/request/**	sprint anterior
Hardening de API — JWT seguro	HS256, secret ≥ 32 bytes por env, access 30 min / refresh 7 dias, validação de assinatura e expiração, sessão STATELESS	security/jwt/JwtTokenProvider.java, JwtAuthenticationFilter.java	sprint anterior
Integridade de payload	HMAC-SHA256 no header X-Signature (POST/PUT), comparação em tempo constante	security/filter/PayloadIntegrityFilter.java	sprint anterior
Erros sem vazamento	GlobalExceptionHandler cobre 12 exceções; nunca expõe stack trace, classe interna nem estrutura de banco	exception/GlobalExceptionHandler.java	sprint anterior
Actuator isolado [novo]	Em produção o Actuator sobe em porta de gestão separada (9090); exposição reduzida a health, info, prometheus; env / beans / heapdump / loggers bloqueados	application.properties, application-prod.properties, SecurityConfig.java	esta sprint
Headers de erro / proxy [novo]	server.error.include-stacktrace=never, include-message=never; forward-headers-strategy=native + remoteip para IP real atrás do proxy	application-prod.properties	esta sprint
Log em arquivo JSON [novo]	Appender rotacionado (riva-backend.json, 50 MB × 7 dias, gzip) para ingestão pelo Loki	logback-spring.xml	esta sprint

Tema	Implementação	Arquivo / classe	Origem
Atualização de dependências [novo]	Correção guiada pelo SCA — ver 2.4	pom.xml	esta sprint
IP real do cliente (anti-spoofing) [novo]	X-Forwarded-For só é confiado quando a conexão direta vem de rede interna/confiável (loopback, RFC1918); um cliente externo não consegue burlar o rate limit nem forjar o IP na trilha de auditoria	security/ClientIpResolver.java	esta sprint (auditoria)
Fail-fast de segredos em produção [novo]	A aplicação não sobe com profile prod se JWT_SECRET, AES_ENCRYPTION_KEY, EMAIL_HASH_SECRET, ADMIN_DEFAULT_PASSWORD (ou HMAC_SECRET, se habilitado) estiverem no valor de fallback de desenvolvimento	config/SecurityStartupValidator.java	esta sprint (auditoria)
Direito à exclusão (LGPD) acessível [novo]	DELETE /api/v1/users/{id} (somente ADMIN) aciona a anonimização irreversível do usuário; antes o serviço existia sem nenhum endpoint que o chamasse	controller/UserController.java, service/AnonymizationService.java	esta sprint (auditoria)

2.4 Correção guiada pelo SCA — atualização de dependências

Na primeira execução do pipeline, a etapa de SCA (Trivy) reprovou o PR com ~30 CVEs HIGH/CRITICAL em dependências transitivas do Spring Boot 3.3.5 (Spring Framework, Spring Security, Tomcat embed, driver PostgreSQL, Spring Data Commons). Correção aplicada no pom.xml:

Componente	Antes	Depois	CVEs resolvidas (exemplos)
spring-boot-starter-parent	3.3.5	3.5.16	CVE-2026-40973 (RCE via tmp dir), CVE-2025-22235
spring-webmvc / expression / core (via BOM)	6.1.14	6.2.19	CVE-2026-41845 (XSS), CVE-2026-41842/41850 (DoS), CVE-2025-41249
spring-security-web / -crypto (via BOM)	6.3.4	6.5.11	CVE-2026-22732 (CRITICAL — bypass de política de segurança), CVE-2025-22228
spring-data-commons (via BOM)	3.3.5	3.5.13	CVE-2026-41716 (DoS por exaustão de cache)
tomcat-embed-core (tomcat.version)	10.1.31	10.1.59	CVE-2024-50379 / CVE-2026-43515 / -65182 / -65905 / -68525 (CRITICAL), CVE-2025-55752 (dir traversal → RCE) e ~15 DoS/inf-disclosure
postgresql (postgresql.version)	42.7.4	42.7.12	CVE-2026-54291 (MITM via downgrade SCRAM), CVE-2025-49146, CVE-2026-42198
springdoc-openapi-starter-webmvc-ui	2.6.0	2.8.9	compatibilidade com Spring Boot 3.5

Validação: ./mvnw clean test → 145/145 testes passando após o upgrade, sem mudança de código de aplicação. O overlay de versões de tomcat e postgresql fica em <properties> do pom.xml,

comentado como override de segurança, para o caso de o BOM ainda não ter alcançado a versão corrigida.

Evidência: as 75 vulnerabilidades listadas no Code Scanning do PR #24, todas fechadas como “fixed” após este upgrade — ver os prints na seção 1.4.

Controle de acesso por perfil

O edital cita perfis genéricos (Brigadista, Gestor, Administrador). No RIVA o RBAC é implementado com três papéis equivalentes, aplicados no SecurityFilterChain:

Papel RIVA	Equivalente no edital	Permissões
ADMIN	Administrador	Tudo, incluindo /api/v1/admin/**, /api/v1/users/** e o Actuator
ANALYST	Gestor	Consultas e comparações de veículos (/api/v1/vehicles/**, /versions/**, /brands/**, ...)
USER	Brigadista / operador	Acesso autenticado básico; sem acesso a dados de administração

Enforcement: SecurityConfig.java (authorizeHttpRequests + @EnableMethodSecurity);

CustomUserDetailsService converte Role em ROLE_<nome>. Teste:

security/SecurityFilterChainIntegrationTest.java.

2.2 Segurança MQTT/TLS para IoT — design de referência

A solução RIVA não possui camada IoT. É uma plataforma de inteligência competitiva (API REST + web Next.js + app React Native/Expo); não há dispositivos, firmware nem broker MQTT no escopo. O tema é coberto como design de referência — como a telemetria seria protegida caso um componente IoT fosse adicionado (ex.: coleta de dados de veículos em pista para o benchmark da Ranger Raptor):

Controle	Especificação de referência
Transporte	MQTT sobre TLS 1.2+ (porta 8883), cifras AEAD apenas, sem downgrade
Autenticação do device	mTLS — certificado X.509 por dispositivo, emitido por CA interna; sem usuário/senha compartilhados
Autorização	ACL por tópico no broker (ex.: device ABC só publica em riva/telemetry/ABC/#)
Identidade	client-id amarrado ao CN do certificado; rejeição de conexões com id divergente
Rotação	Validade curta (90 dias) + rotação automatizada; CRL/OCSP para revogação
Payload	Assinatura HMAC no payload (reuso do padrão do PayloadIntegrityFilter) e schema validation na ingestão
Rede	Broker em subnet privada; ingestão para a API só via fila interna, nunca exposição direta

2.3 IaC Security

Artefato	Boas práticas aplicadas	Verificação
Dockerfile	Multi-stage (build isolado do runtime); imagem-base fixada por versão (eclipse-temurin:21-jre-alpine, sem :latest); apk upgrade do SO-base; usuário não-root (riva, /sbin/nologin); sem segredos embutidos; healthcheck no liveness probe	Trivy config no job iac-container-scan

Artefato	Boas práticas aplicadas	Verificação
.dockerignore	Exclui .git, .env*, *.p12 / *.pem / *.key, docs/, observability/ do contexto de build	—
docker-compose.yml	Postgres só para dev local; healthcheck; sem portas desnecessárias	Trivy config
application-prod.properties	Conexão ao Postgres Azure com sslmode=require (TLS obrigatório); credenciais 100% por variável de ambiente	revisão + SECURITY.md §11
security.yml	permissions mínimas por workflow; concurrency para cancelar runs obsoletas; versões de actions fixadas	Trivy config + Dependabot (github-actions)

Verificação: o Dockerfile é analisado pelo Trivy config e a imagem pelo Trivy image no job iac-container-scan (relatórios no artifact trivy-reports); o comando docker run --rm riva-backend:local id confirma o processo como usuário riva (Apêndice B).

2.5 Auditoria interna do código e correções adicionais

Antes da entrega, todo o código de segurança passou por uma auditoria interna completa (autenticação, RBAC e filtros; proteção de dados e LGPD; logging, CI e observabilidade). Os achados acionáveis foram corrigidos no commit f9d6d7d (PR #42, já incorporado à branch main); o que não foi corrigido está registrado como limitação conhecida no SECURITY.md §14.

Achado (severidade)	Correção aplicada	Arquivo / classe	Verificação
X-Forwarded-For confiado sem validação (crítico): permitia burlar o rate limit do login e forjar o IP na auditoria	O header só é honrado quando a conexão direta vem de rede interna/confiável	security/ClientIpResolver.java, MdcFilter, RateLimitFilter	RateLimitIntegrationTest, MdcFilterTest
Segredos de desenvolvimento aceitos em produção (crítico)	Startup falha em profile prod se algum segredo estiver no valor de fallback	config/SecurityStartupValidator.java	SecurityStartupValidatorTest (8 testes)
Exclusão LGPD inalcançável, sem endpoint (crítico / compliance)	DELETE /api/v1/users/{id} (ADMIN); usuário inexistente devolve 404 padronizado	controller/UserController.java, service/AnonymizationService.java	UserControllerIntegrationTest (3 testes)
Logs duplicados quando os perfis dev e obs estavam ativos juntos (médio)	Condições de springProfile mutuamente exclusivas: dev/test, dev/test + obs e prod	logback-spring.xml	Execução da suíte: uma linha por evento

Achado (severidade)	Correção aplicada	Arquivo / classe	Verificação
Token válido de usuário removido resultava em 500 não tratado (médio)	UsernameNotFoundException tratada: segue sem autenticar e a cadeia responde 401 padronizado	security/jwt/JwtAuthenticationFilter.java	SecurityFilterChainIntegrationTest
Mapas por IP sem limpeza: crescimento indefinido de memória (médio)	Limpeza agendada dos buckets e do histórico de falhas de login	RateLimitFilter, LoginAttemptService	LoginAttemptServiceTest
Workflows de CI sem permissions explícitas (baixo)	permissions: contents: read (privilegio mínimo)	.github/workflows/develop.yml, production.yml	CI verde na PR #42

Validação: ./mvnw clean test → 156/156 testes passando (145 anteriores + 8 de SecurityStartupValidatorTest + 3 de UserControllerIntegrationTest); pipeline de segurança (Gitleaks, Semgrep, Trivy, Dependency-Check) verde no commit da main.

Limitações conhecidas (documentadas, não corrigidas): HMAC de payload sem proteção a replay; InputSanitizer.sanitize() não plugado globalmente (a rejeição via @SafeText + @Pattern está ativa); gate de CI parcial (apenas o Secret Scanning bloqueia o merge); revogação de access token não é imediata (máx. 30 min).

ATIVIDADE 3

Compliance, Riscos e Segurança Contínua

PESO 2,5

Objetivo: demonstrar que o sistema segue boas práticas, normas e políticas de segurança.

3.1 Revisão final de riscos — STRIDE + DevSecOps

Componente	Ameaça (STRIDE)	Mitigação implementada	Camada DevSecOps que reforça	Status
API REST	Spoofing de usuário	JWT HS256 + validação de assinatura; sessão stateless	SAST; testes de auth	Mitigado
API REST	Tampering de payload	HMAC-SHA256 X-Signature (prod); Bean Validation + @SafeText	SAST (injection); Semgrep owasp-top-ten	Mitigado
API REST	Repudiation	Trilha de auditoria AuditLog (transação REQUIRES_NEW)	Log JSON em arquivo + Loki (Atividade 4)	Mitigado
Dados (User)	Information Disclosure	AES-256-GCM em repouso; blind index; @JsonIgnore; DTOs de resposta	Secret Scanning; SCA (libs de cripto)	Mitigado
Autenticação	Denial of Service (brute force)	LoginAttemptService (janela 5 min) + rate limit 10/min no login	Alertas AuthLoginFailureSpike / RateLimitRejectionSpike	Mitigado + monitorado
API REST	Denial of Service (flooding)	Bucket4j 60/min por IP; 429 + Retry-After	Alerta RateLimitRejectionSpike; runbook DoS	Mitigado + monitorado
Autorização	Elevation of Privilege	RBAC no SecurityFilterChain + @EnableMethodSecurity	SAST; teste SecurityFilterChainIntegrationTest	Mitigado
Dependências	Elevation via CVE em lib	Upgrade Spring Boot 3.3.5 → 3.5.16 + overrides (ver 2.4)	SCA (Trivy + Dependency-Check) + Dependabot	1º ciclo aplicado; contínuo
Pipeline / supply chain	Tampering no build	permissions mínimas; actions fixadas; Dependency Review	Trivy config + Dependency Review	Contínuo
Infra / imagem	Tampering / Misconfig	Dockerfile endurecido; Actuator isolado; TLS no banco	Trivy config + image	Mitigado + monitorado
Observabilidade	Information Disclosure (log com PII)	Política “sem senha/token/PII no log”; só username em eventos	Revisão de PR + Semgrep secrets	Mitigado

3.2 Mapeamento com normas e boas práticas

OWASP ASVS 4.0 — nível 2 (controles-chave)

Cap.	Requisito	Evidência RIVA	Status
V2 Autenticação	Senha com hash forte + política	BCrypt strength 12; RegisterRequest com @Size / @Pattern	OK

Cap.	Requisito	Evidência RIVA	Status
V2 Autenticação	Proteção contra brute force	LoginAttemptService + rate limit no login	OK
V3 Sessão	Tokens stateless, expiração curta, sem sessão no servidor	JWT access 30 min / refresh 7 dias; SessionCreationPolicy.STATELESS	OK
V4 Controle de acesso	RBAC deny-by-default	anyRequest().authenticated() + matchers por papel	OK
V5 Validação	Validação positiva (whitelist) e sanitização	@Pattern whitelist + InputSanitizer	OK
V7 Erros e logs	Sem stack trace ao cliente; log estruturado	GlobalExceptionHandler; logback-spring.xml JSON	OK
V8 Proteção de dados	Cripto em repouso de dado pessoal	AES-256-GCM no email	OK
V9 Comunicação	TLS em trânsito	HTTPS documentado (SECURITY.md §12); Postgres sslmode=require	Parcial (TLS da API só em produção)
V10 Código malicioso	SAST + SCA no CI	Semgrep + Dependency-Check / Trivy	OK
V12 Arquivos / recursos	Sem path traversal	InputSanitizer (padrões ../, %2e%2e, /etc/passwd)	OK
V13 APIs	Rate limiting; autenticação em todos os endpoints não públicos	RateLimitFilter; matchers	OK
V14 Configuração	Build reproduzível; sem segredo em repo; headers seguros	Docker multi-stage; env vars; Gitleaks	OK

OWASP API Security Top 10 (2023)

ID	Risco	Tratamento RIVA	Status
API1	Broken Object Level Authorization	Endpoints por papel; IDs internos (surrogate keys); sem exposição de objeto de outro usuário	OK
API2	Broken Authentication	JWT validado, expiração, brute-force detection	OK
API3	Broken Object Property Level Authorization	DTOs de resposta separados das entidades; @JsonIgnore no password	OK
API4	Unrestricted Resource Consumption	Rate limit Bucket4j; @Size em campos e listas (máx. 50 atributos / 200 chars)	OK
API5	Broken Function Level Authorization	RBAC no filter chain + method security	OK
API6	Unrestricted Access to Sensitive Business Flows	Rate limit no login; auditoria de MASS_QUERY	OK
API7	Server-Side Request Forgery	Sem fetch de URL fornecida pelo cliente no backend; scraping isolado no serviço de IA	OK (n/a no backend)
API8	Security Misconfiguration	Trivy config; Actuator isolado; CORS restritivo; erros genéricos	OK
API9	Improper Inventory Management	OpenAPI/Swagger versionado (/v3/api-docs); ambientes dev / prod separados	OK

ID	Risco	Tratamento RIVA	Status
API10	Unsafe Consumption of 3rd-party APIs	SCA nas libs; validação de schema na borda	Contínuo

OWASP Mobile Top 10 (2024) — app React Native / Expo

O app é responsabilidade da disciplina de Mobile. Abaixo o mapa de responsabilidade compartilhada — o que o backend garante e o que o time mobile precisa garantir:

ID	Risco	Backend garante	Time mobile precisa garantir
M1	Credential Usage	—	Não embutir credenciais no bundle; usar fluxo de login
M2	Supply Chain	SCA no backend	npm audit / Dependabot no repo mobile
M3	Auth / Authz	JWT + RBAC no servidor	Não confiar em checagem local; renovar via /auth/refresh
M4	Input/Output Validation	Validação server-side	Sanitizar entrada antes de exibir (anti-XSS em WebView)
M5	Insecure Communication	TLS obrigatório; CORS	Certificate pinning; recusar HTTP
M6	Privacy	Pseudonimização; LGPD	Não logar PII no device; pedir consentimento
M7	Binary Protections	—	Ofuscação; detectar root / jailbreak se aplicável
M8	Security Misconfiguration	Headers; erros genéricos	Desabilitar debug no build de release
M9	Insecure Data Storage	—	expo-secure-store para o token, nunca AsyncStorage puro
M10	Insufficient Cryptography	AES-256-GCM no servidor	Não implementar cripto própria; usar APIs da plataforma

LGPD (Lei 13.709/2018)

Detalhado no SECURITY.md §8. Resumo:

Item	Tratamento
Dados pessoais tratados	username, email (cifrado), IP em log de auditoria
Base legal	Execução de contrato / legítimo interesse (ferramenta corporativa)
Minimização	Só o necessário para autenticação e auditoria; busca de veículos não tem PII
Retenção	Tokens: 30 min / 7 dias · AuditLog: 90 dias (job de anonimização diário) · User: enquanto conta ativa
Direito à exclusão (Art. 18, V)	DELETE /api/v1/users/{id} (ADMIN) → AnonymizationService.anonymizeUser(id) — substituição irreversível, preserva integridade referencial
Pseudonimização p/ ML e BI	email_hash (HMAC) como pseudônimo estável; AuditLog pós-retenção com user_id / ip nulos; segregação de chave EXPORT_SECRET
Segurança do tratamento (Art. 46)	Cripto em repouso e trânsito, RBAC, auditoria, logging sem PII
Telemetria e localização	O backend RIVA não coleta localização nem telemetria de dispositivos. Se o app mobile ou um componente IoT passarem a enviá-las: minimização e finalidade declarada, consentimento explícito e revogável, precisão reduzida (arredondamento/geohash) quando possível, cifragem em trânsito e em repouso, retenção curta e pseudonimização (mesma política do AuditLog)

3.3 Plano de segurança contínua**Rotina de revisão de dependências**

Quando	Ação	Responsável
Semanal (automático)	Dependabot abre PRs (Maven, Actions, Docker); security.yml roda SCA completo	CI
A cada PR	Dependency Review comenta dependência vulnerável nova (warn-only)	CI + revisor
Semanal (humano)	Triagem dos achados de SCA: classificar CVSS, alcançabilidade, exploit; agir ou registrar risco aceito com prazo	Responsável de segurança
Release	Confirmar que não há CVE ≥ 7 pendente sem mitigação	Líder técnico

Rotina de testes de segurança

Quando	Ação
A cada commit / PR	SAST (Semgrep), Secret Scanning (Gitleaks), 156 testes automatizados
A cada PR	Checklist de segurança no template de PR (.github/pull_request_template.md)
Por sprint	Pentest interno leve: OWASP ZAP baseline contra homologação; revisão manual de um fluxo crítico
Trimestral	Revisão do modelo de ameaças (STRIDE) e do mapeamento ASVS

Checklist de PR (arquivo .github/pull_request_template.md):

- Entrada nova validada (@Valid + @SafeText onde aplicável)
- Endpoint novo coberto por regra de RBAC explícita no SecurityConfig
- Nenhum dado sensível em log / resposta / URL (senha, token, e-mail, PII)
- Segredo novo via variável de ambiente e documentado no SECURITY.md §11
- Teste automatizado cobrindo o caminho de erro / negação de acesso
- Pipeline DevSecOps verde (ou achados triados e justificados)

Rotina de auditoria de permissões

Quando	Ação
Mensal	Listar usuários com papel ADMIN; confirmar necessidade; remover excedentes
Mensal	Revisar matchers do SecurityConfig vs. endpoints existentes (nenhum endpoint sem regra)
Por sprint	Revisar acessos ao repositório GitHub e aos segredos do Actions; remover ex-membros
Trimestral	Revisar validade / rotação de JWT_SECRET, AES_ENCRYPTION_KEY, EMAIL_HASH_SECRET, HMAC_SECRET

Rotina de backup e recuperação (iniciada na Fase 3)

Item	Definição
Escopo	PostgreSQL de produção (Azure Database for PostgreSQL)
Backup	Automático da Azure — full diário + WAL contínuo (Point-in-Time Restore)
Retenção	7 dias de PITR (configurável até 35)
RPO	≤ 5 min (perda máxima aceitável de dados)
RTO	≤ 1 h (tempo máximo para restaurar o serviço)
Teste de restore	Mensal — restaurar snapshot num banco temporário e rodar smoke test da API
Segredos	Guardados no secret store (Actions Secrets / Key Vault); export cifrado offline trimestral
Auditoria	Confirmar que AuditLog e dados cifrados voltam íntegros após restore (GCM tag válida)

3.4 Checklist de conformidade consolidado

Domínio	Item	Status	Evidência
Pipeline	Secret scanning em todo commit e histórico	OK	security.yml job secret-scan
Pipeline	SAST em todo PR	OK	job sast
Pipeline	SCA + Dependabot ativos	OK	job sca + dependabot.yml
Pipeline	IaC e imagem escaneadas	OK	job iac-container-scan
Pipeline	PR sinaliza dependência vulnerável nova	OK	job dependency-review (warn-only)
Código	Entrada validada e sanitizada (4 camadas)	OK	InputSanitizer, DTOs, 156 testes
Código	AuthN JWT + AuthZ RBAC	OK	security/**, SecurityConfig
Código	Cripto em repouso de PII	OK	crypto/AesEncryptor
Código	Erros sem vazamento de internals	OK	GlobalExceptionHandler
Infra	Imagem não-root, base fixada, multi-stage	OK	Dockerfile + Trivy
Infra	Actuator isolado em produção	OK	application-prod.properties
Infra	TLS no banco (sslmode=require)	OK	application-prod.properties
Infra	TLS na API	Pendente	documentado; ativar em produção (SECURITY.md §12)
Compliance	OWASP ASVS nível 2 mapeado	OK	§3.2
Compliance	OWASP API Top 10 mapeado	OK	§3.2
Compliance	OWASP Mobile Top 10 mapeado	OK	§3.2
Compliance	LGPD — direitos, retenção, pseudonimização	OK	SECURITY.md §8
Contínuo	Rotina de revisão de dependências	OK	§3.3
Contínuo	Rotina de testes de segurança	OK	§3.3
Contínuo	Rotina de auditoria de permissões	OK	§3.3
Contínuo	Rotina de backup + teste de restore	OK	§3.3
Runtime	Métricas + alertas de segurança	OK	observability/ (Atividade 4)
Runtime	Plano de resposta a incidentes	OK	RUNBOOK_RESPOSTA_INCIDENTES.md

ATIVIDADE 4

Observabilidade, Monitoramento e Resposta

PESO 2,0

Objetivo: mostrar como o sistema detecta, registra e responde a incidentes. Artefatos executáveis em observability/.

4.1 Plano de monitoramento

Logs estruturados

Aspecto	Definição
Formato	JSON (LogstashEncoder) nos profiles prod / obs; console legível em dev / test
Campos fixos	timestamp, level, logger_name, message, thread_name, application
Correlação (MDC)	trace_id (UUID por request), user_id, client_ip — populados pelo MdcFilter (1º filtro)
Destino	stdout (ELK/Datadog no ambiente real) e arquivo rotacionado logs/riva-backend.json → Promtail → Loki
Rotação	50 MB por arquivo, 7 dias, compactação gzip, cap de 1 GB
É logado	login OK/falho, alterações críticas de conta (registro e anonimização de usuário — LGPD), brute force detectado, rate limit, HMAC inválido, 404 de scan, acesso negado
Nunca é logado	senha, hash de senha, token JWT, e-mail ou outro dado pessoal (só username em eventos)

Eventos monitorados (regras já existentes no código — ver SECURITY.md §10):

Regra	Origem	Nível
5+ falhas de login do mesmo IP em 5 min	AuthService + LoginAttemptService	ERROR
Falha de login isolada	AuthService	WARN
IP a 80% / acima do rate limit	RateLimitFilter	WARN
Assinatura HMAC inválida	PayloadIntegrityFilter	WARN
404 em endpoint inexistente (scan)	GlobalExceptionHandler	WARN
Acesso não autenticado / negado	JwtAuthenticationEntryPoint / JwtAccessDeniedHandler	WARN

Métricas e alertas

- Coleta: Spring Boot Actuator + Micrometer → /actuator/prometheus (porta de gestão 9090 em produção) → Prometheus (scrape 15 s).
- Métricas-chave: http_server_requests_seconds_* (taxa, status, latência p50/p95/p99), logback_events_total{level="error"}, jvm_memory_used_bytes, process_cpu_usage, hikaricp_connections_active.

Alerta	Condição	Severidade	Playbook
RivaBackendDown	up == 0 por 1m	critical	Runbook — Serviço indisponível

Alerta	Condição	Severidade	Playbook
RivaHealthDegraded	app no ar (up) sem sinalizar readiness por 5m	warning	investigar readiness (banco / dependências)
HighHttp5xxRate	> 5% de 5xx por 5m	critical	Runbook — Erros 5xx
ErrorLogSurge	> 20 logs ERROR / 10m	warning	Runbook — Surto de erros
AuthLoginFailureSpike	> 30 falhas de login/min por 3m	warning (seg.)	Runbook — Brute force
RateLimitRejectionSpike	> 120 respostas 429/min por 3m	warning (seg.)	Runbook — Flooding/DoS
ForbiddenResponseSpike	> 60 respostas 403/min por 5m	info (seg.)	triagem
NotFoundScanSpike	> 100 respostas 404/min por 5m	info (seg.)	triagem
HighLatencyP99	p99 > 1s por 10m	warning	investigação de performance
JvmHeapPressure	heap > 90% por 10m	warning	investigar leak / ajustar memória
HighCpuUsage	CPU > 85% por 10m	warning	investigar carga

- Roteamento: Alertmanager com receiver dedicado “segurança” (webhook/e-mail a preencher) e inhibit_rule para silenciar derivados quando o serviço está fora do ar.
- SLOs de referência: disponibilidade 99,5% mensal; p99 < 500 ms; erro 5xx < 1%.

Cobertura por componente (API, mobile, IoT e ML)

Escopo do edital: métricas e alertas para API, mobile, IoT e ML. A API está implementada e evidenciada; os demais componentes têm o plano de monitoramento definido como referência, para quando forem integrados.

Componente	Métricas e alertas
API (implementado)	Actuator + Micrometer: taxa, status e latência p50/p95/p99, JVM, CPU, pool de conexões; logs JSON com trace_id/user_id/client_ip; 11 regras de alerta (tabela acima) no Prometheus/Alertmanager
Mobile (referência — time mobile)	Crash rate e ANR (ex.: Firebase Crashlytics ou Sentry); picos de 401/403 por versão do app e taxa de refresh de token, visíveis nas métricas da API; alerta se crash-free sessions < 99%
IoT (referência — sem camada IoT no escopo)	Métricas do broker MQTT: clientes conectados, mensagens/s, falhas de handshake TLS/mTLS e de ACL; alertas para queda de dispositivos, pico de falhas de autenticação e dispositivo silencioso além do limite
ML (referência — serviço de IA)	Latência e taxa de erro da inferência, proporção de respostas “Não disponível”, drift da distribuição das entradas; alertas para erro > 5%, p95 acima do limite e queda abrupta de qualidade

Dashboards

Stack em observability/docker-compose.observability.yml: Prometheus + Alertmanager + Loki + Promtail + Grafana. Dashboard provisionado “RIVA - Segurança & Observabilidade” (riva-observability.json) com as linhas: Visão geral · HTTP · Segurança (falhas de login, 429, 403/404, painel de logs de eventos de segurança, painel de logs ERROR) · Recursos.

```
docker compose -f observability/docker-compose.observability.yml up -d
Grafana      http://localhost:3001  (admin / admin)
Prometheus   http://localhost:9091
Alertmanager http://localhost:9093
```

4.2 Plano de resposta a incidentes

Ciclo Detecção → Análise → Contenção → Erradicação → Recuperação → Pós-morte, com papéis, severidades (SEV1–SEV3), canais e playbooks por tipo de evento (brute force, flooding, 5xx, serviço fora do ar, segredo vazado, dependência vulnerável) detalhados no arquivo docs/RUNBOOK_RESPOSTA_INCIDENTES.md.

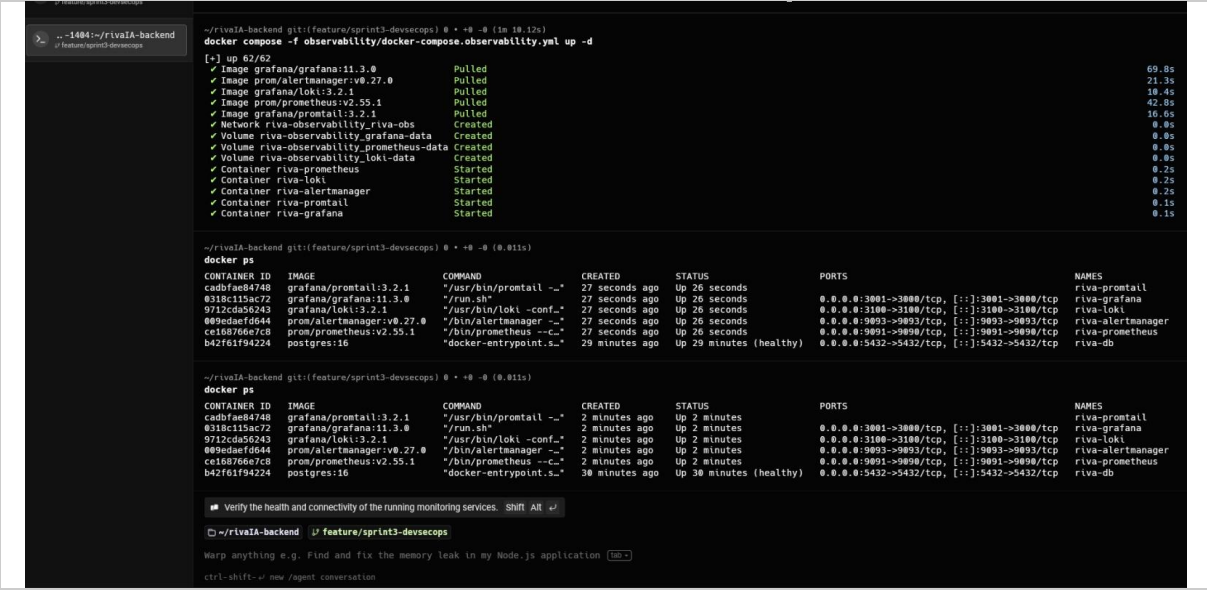
Fase	Objetivo	Responsável
Detecção	Reconhecer que há um incidente (alerta, log, denúncia)	On-call
Análise	Classificar severidade, escopo e vetor	On-call + Sec
Contenção	Parar o dano em curso (bloquear IP, revogar token, isolar)	On-call
Erradicação	Remover a causa (corrigir bug, girar segredo, subir patch)	Dev + Sec
Recuperação	Restaurar serviço normal e confirmar integridade	Dev
Pós-morte	Documentar linha do tempo, causa-raiz e ações	Sec (facilita)
Severidade	Definição	1ª resposta
SEV1	Vazamento de dado pessoal, RCE, comprometimento de credencial de produção, serviço fora do ar	15 min
SEV2	Ataque ativo contido (brute force, flooding), vulnerabilidade crítica explorável	1 h
SEV3	Varredura, tentativa isolada, vulnerabilidade sem exploração observada	1 dia útil

4.3 Evidências coletadas

Stack de observabilidade executado localmente (Postgres + Prometheus + Alertmanager + Loki + Promtail + Grafana), com carga de teste gerada manualmente (logins inválidos e rajada de requisições) para validar a coleta de métricas e as regras de alerta ponta a ponta.

Infraestrutura no ar: docker compose up — todos os containers do stack (Prometheus, Grafana, Loki, Promtail, Alertmanager) e o Postgres de desenvolvimento saudáveis.

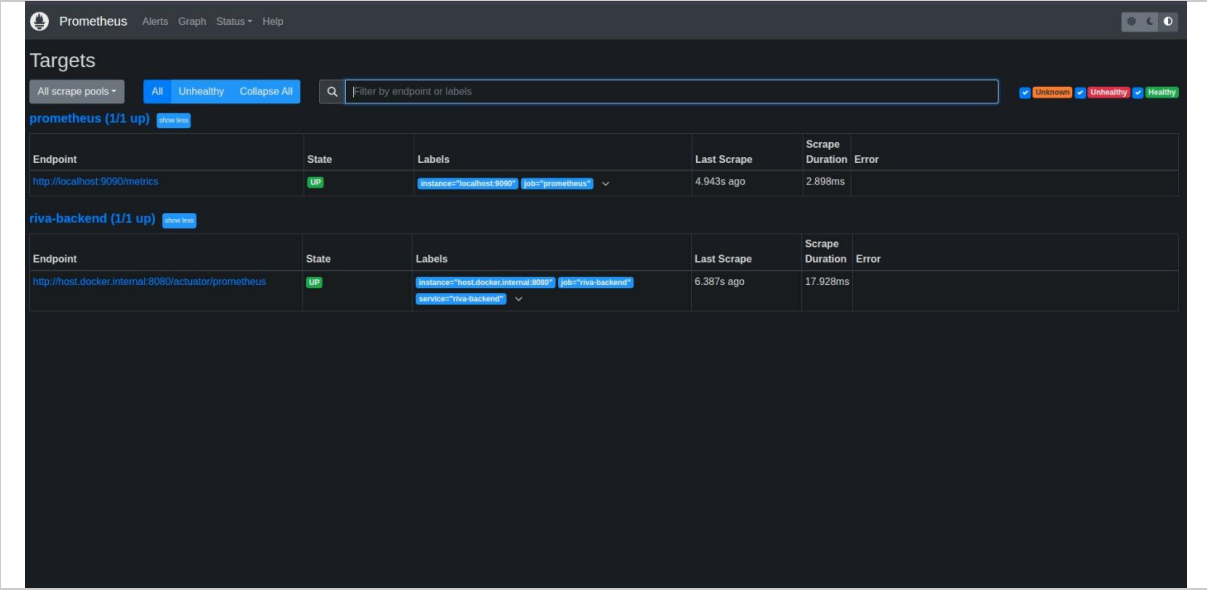
EVIDÊNCIA / SCREENSHOT



Evidência — containers do stack de observabilidade no ar (docker compose ps).

Scrape do Atuator funcionando: o alvo riva-backend passou de DOWN (porta 9090, incorreta em ambiente local) para UP na porta 8080, correção aplicada durante a validação.

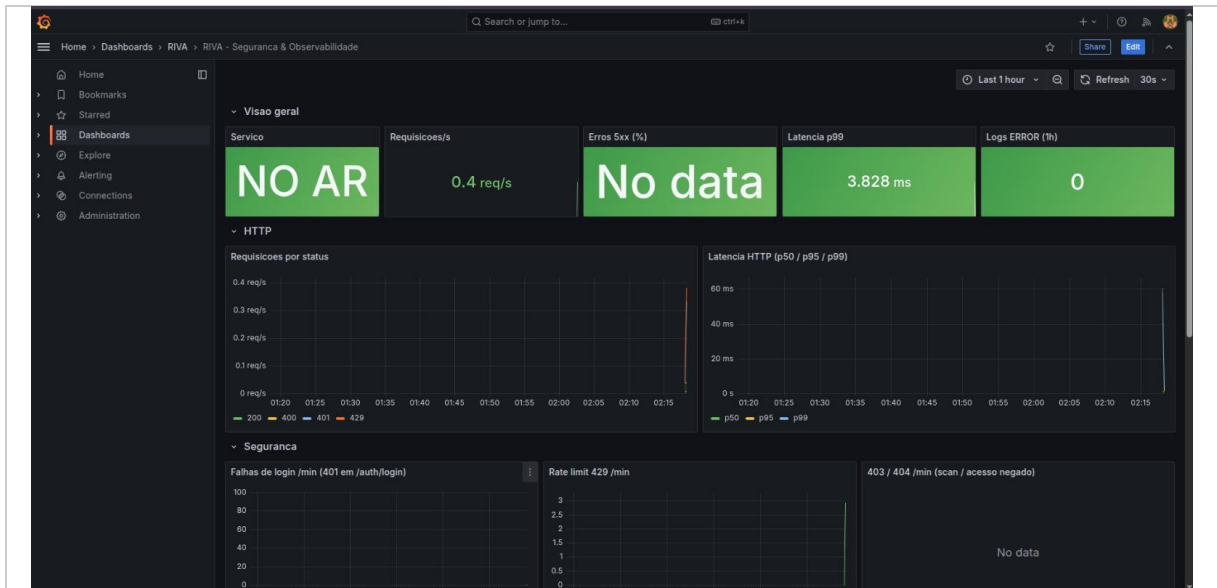
EVIDÊNCIA / SCREENSHOT



Evidência — Prometheus coletando métricas reais do riva-backend (/actuator/prometheus).

Dashboard Grafana com dados reais: requisições/s, latência p99, uso de recursos (heap, CPU, threads) refletindo o tráfego gerado nos testes.

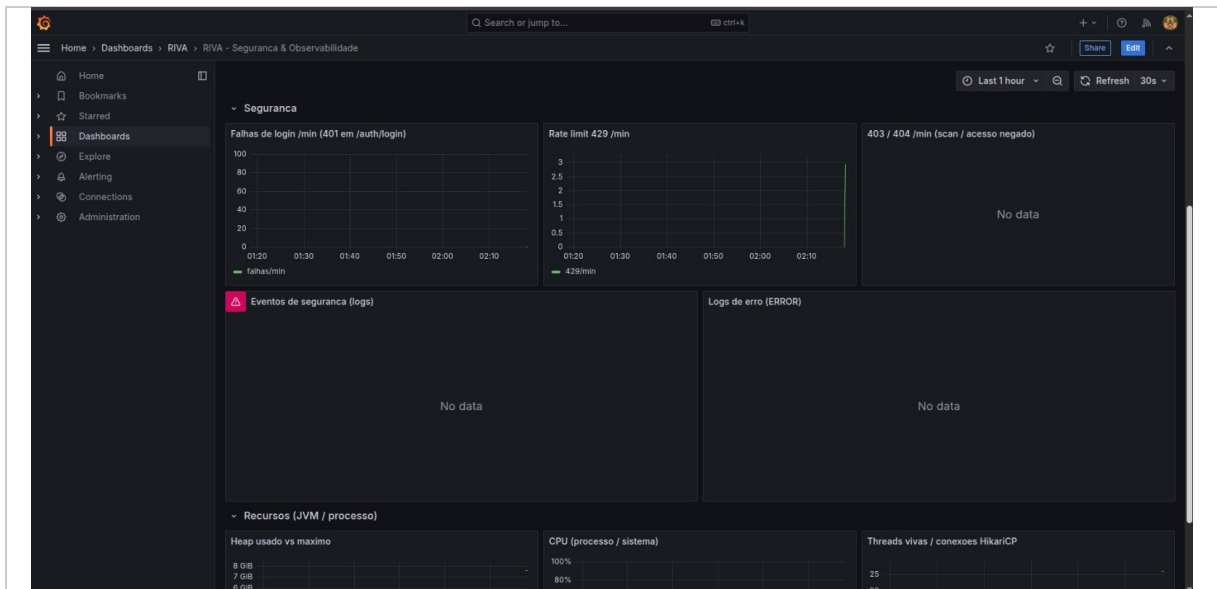
EVIDÊNCIA / SCREENSHOT



Evidência — painel “Visão geral” do dashboard RIVA - Segurança & Observabilidade, com métricas reais coletadas.

Painel de segurança: pico real registrado no gráfico “Rate limit 429/min” após a rajada de requisições de teste, confirmando que o RateLimitFilter e a métrica correspondente estão instrumentados corretamente.

EVIDÊNCIA / SCREENSHOT

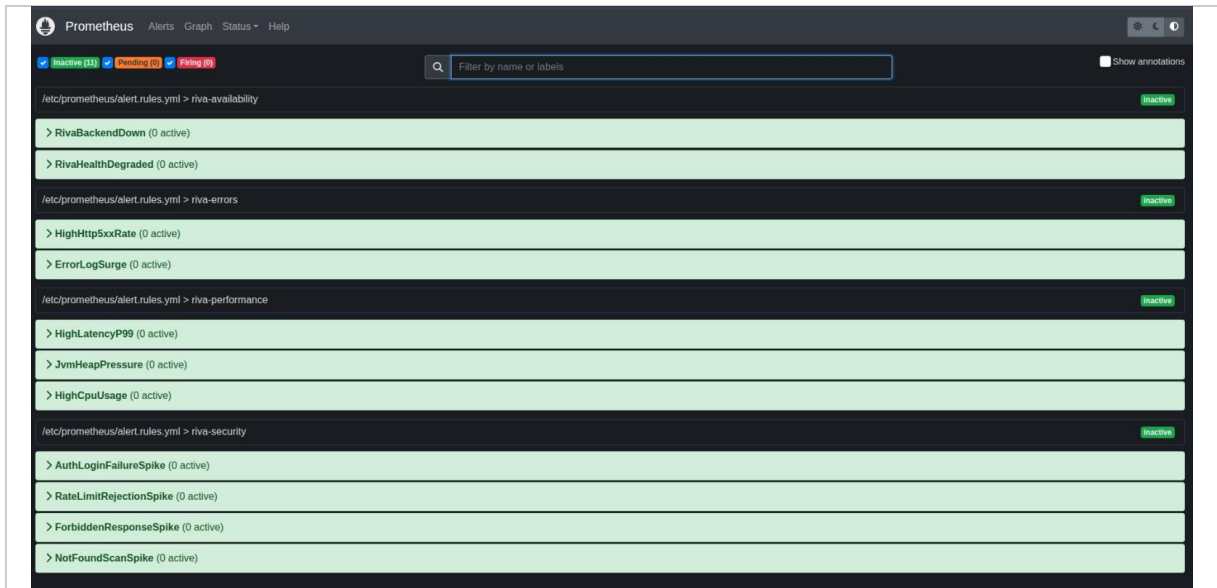


Evidência — painel “Segurança”: pico de respostas 429 (rate limit) capturado durante o teste de carga.

Regras de alerta carregadas: as 11 regras definidas em alert.rules.yml (disponibilidade, erros, performance e segurança) ativas e sendo avaliadas continuamente pelo Prometheus. Nenhuma em Firing no momento do print — a carga de teste (rajada de segundos) não sustentou a janela mínima

("for") exigida por essas regras, o que é o comportamento esperado e correto de um alerta bem calibrado contra ruído.

EVIDÊNCIA / SCREENSHOT



Evidência — as 11 regras de alerta carregadas no Prometheus (Alerts), todas avaliadas e Inactive no momento do teste.

Exemplo de log estruturado (JSON): saída real da aplicação durante o teste de rate limit, evidenciando o formato JSON, o trace_id de correlação e a ausência de dados sensíveis (senha/token) no log.

EVIDÊNCIA / SCREENSHOT



Evidência — log JSON estruturado do RateLimitFilter (nível WARN) durante a rajada de teste.

Apêndice A — Índice de evidências por atividade

Arquivo	Atividade
.github/workflows/security.yml	1
.github/dependabot.yml	1
.gitleaks.toml	1
Dockerfile, .dockerignore	2
src/main/java/com/ford/riva/config/SecurityConfig.java (diff)	2
src/main/resources/application.properties, application-prod.properties (diff)	2
src/main/resources/logback-spring.xml (diff)	2, 4
SECURITY.md	2, 3
Este documento §3	3
observability/**	4
docs/RUNBOOK_RESPOSTA_INCIDENTES.md	4
src/main/java/com/ford/riva/security/ClientIpResolver.java, config/SecurityStartupValidator.java, controller/UserController.java	2
SECURITY.md §14 (limitações conhecidas)	2, 3

Apêndice B — Rodar tudo localmente

```
# 1. Testes + build
./mvnw -B clean test

# 2. App com observabilidade (gera logs/riva-backend.json; Actuator na 8080 local, 9090 só em prod)
docker compose up -d
SPRING_PROFILES_ACTIVE=dev,obs ./mvnw spring-boot:run

# 3. Stack de monitoramento
docker compose -f observability/docker-compose.observability.yml up -d

# 4. Imagem de producao endurecida
docker build -t riva-backend:local .
docker run --rm riva-backend:local id          # confirma usuario nao-root riva

# 5. Rodar o pipeline de seguranca localmente (opcional)
gitleaks detect --config .gitleaks.toml --redact
docker run --rm -v "$PWD:/src" semgrep/semgrep semgrep scan --config p/java --config p/owasp-top-ten /src
docker run --rm -v "$PWD:/src" aquasec/trivy fs --scanners vuln,misconfig /src
```

Apêndice C — Glossário

- SAST — análise estática do código-fonte.
- SCA — análise de composição de software (CVE em dependências).
- IaC — infraestrutura como código.
- SARIF — formato padrão de troca de achados de segurança.
- mTLS — TLS mútuo (cliente e servidor autenticam-se por certificado).
- RPO / RTO — perda máxima de dados / tempo máximo de recuperação.
- PITR — restauração para um ponto no tempo.